Limbajul de asamblare. Familia Intel

Elemente de bază (1)

- fiecare instrucțiune - o singură operație
  - unei instrucțiuni într-un limbaj de nivel înalt îi corespund în general mai multe instrucțiuni în limbaj de asamblare
- operanzi
  - constante
  - locații de memorie
  - regiștri ai procesorului

Elemente de bază (2)

- variabilele sunt accesate prin specificarea adresei lor în memorie
- regiștrii procesorului nu sunt accesibili din limbajele de nivel înalt
  - dar compilatorul îi poate folosi temporar în locul variabilelor
  - avantaj - viteză de acces mai mare

Procesoare Intel (32 biți) - regiștri

	31	15 8	7 0	
EAX		AH	AL	AX
EBX		BH	BL	BX
ECX		CH	CL	CX
EDX		DH	DL	DX
ESI				SI
EDI				DI
EBP				BP
ESP				SP

Dimensiunea regiștrilor

- EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP - 32 biți
- AX, BX, CX, DX, SI, DI, BP, SP - 16 biți
  - fiecare reprezintă jumătatea mai puțin semnificativă a registrului corespunzător pe 32 biți
- AH, AL, BH, BL, CH, CL, DH, DL - 8 biți
  - jumătățile unor regiștri pe 16 biți

Regiștri speciali

- ESP
  - reține adresa vârfului stivei programului
  - actualizat prin instrucțiunile de lucru cu stiva
- EBP
  - folosit de compilatoare în apelurile de funcții
- acești regiștri în general nu sunt utilizați decât pentru scopurile menționate mai sus

Utilizarea regiștrilor

- memorarea de valori numerice - la fel ca variabilele de memorie
  - toți regiștrii anteriori
- formarea adreselor de memorie
  - doar regiștrii pe 32 biți
  - în expresii mai complexe, care pot conține mai mulți regiștri și constante

Formarea adreselor - variante (1)

- constantă
  - ex.: `[100]`
- valoarea unui registru
  - ex.: `[EAX]`
- suma dintre valoarea unui registru şi o constantă
  - ex.: `[EBX+5]`

Formarea adreselor - variante (2)

- suma dintre valorile a doi regiștri
  - ex.: `[ECX+ESI]`
- suma dintre valorile a doi regiștri şi o constantă
  - ex.: `[EDX+EBP+14]`
- un registru poate fi înmulțit cu 2/4/8
  - ex.: `[EAX+EDI*2]`, `[ECX+EDX*4+5]`

Indicatori de condiţii (1)

- aduc informații suplimentare despre rezultatele operațiilor tocmai executate
  - separat de rezultatul propriu-zis
- valorile lor pot fi testate prin program
- singura formă de a implementa structuri de control în limbajul de asamblare

Indicatori de condiţii (2)

- cel mai des utilizați

Carry (C)

Overflow (O)

Zero (Z)

  – 1 dacă rezultatul ultimei operații este 0; altfel 0

Sign (S)

  – 0 - rezultat pozitiv; 1 - rezultat negativ

Visual C++

- mediu de programare în care se pot intercala ușor instrucțiuni în limbaj de asamblare
  - doar pentru programe pe 32 biți
- programul în ansamblul său este scris în C/C++
- instrucțiunile în limbaj de asamblare pot accesa regiștrii procesorului, dar și variabilele declarate în C/C++

Sintaxă

- cuvânt-cheie: `_asm`
- poate fi urmat de
  - o singură instrucțiune
  - un bloc de instrucțiuni
  - delimitarea blocurilor - la fel ca în C/C++
  - separatorul între instrucțiuni în blocuri
    - linie nouă (recomandat pentru claritate)
    - caracterul `";"`

Exemple

```
_asm mov eax,5;

int a;
_asm {
  mov eax,5
  mov a,10
  }
```

Instrucţiuni aritmetice

Operanzi (1)

- 3 categorii
  - regiștri de uz general
  - locații de memorie (variabile)
  - constante
- reguli generale (pot exista excepții)
  - operanzii au aceeași dimensiune
  - primul/singurul operand este și destinația unde se depune rezultatul

Operanzi (2)

- operații binare
  - primul operand nu poate fi o constantă
    - deoarece nu ar putea memora rezultatul
  - nu este permis ca ambii operanzi să fie locații de memorie
    - la procesoarele Intel
    - se forțează utilizarea regiștrilor
    - viteză mai mare

Atribuire

- sintaxa

```
mov destinatie,sursa
```

- exemple

```
mov eax,ebx
mov cx,5
mov bl,[eax]
mov [esi],edx
```

Dimensiunea operanzilor (1)

- un registru are mereu aceeași dimensiune
- dar aceeași constantă poate ocupa 1/2/4 octeți
  - în funcție de context
- la fel și un operand adresă de memorie
- regulă: dacă unul din operanzi este un registru, celălalt are aceeași dimensiune

Dimensiunea operanzilor (2)

- dacă nici un operand nu este registru, trebuie precizată explicit dimensiunea lor
- cuvinte cheie
  - `byte ptr` - operand pe 1 octet
  - `word ptr` - operand pe 2 octeți
  - `dword ptr` - operand pe 4 octeți

Exemple

`mov al,5` // 1 octet - dimensiunea registrului `al`

`mov bx,[esi]` // 2 octeţi - dimensiunea registrului `bx`

`mov byte ptr [ecx],5` // 1 octet

`mov word ptr [ecx],5` // 2 octeţi

`mov dword ptr [ecx],5` // 4 octeţi

Dimensiunea operanzilor (3)

- Visual C++: dacă locația de memorie este o variabilă declarată în C/C++, nu mai trebuie precizată explicit dimensiunea
  - compilatorul o cunoaște din declarație
  - și deci poate genera instrucțiunea corectă
- atenție
  - regula operanzilor de dimensiuni egale rămâne valabilă

Exemple

```
int a;
_asm mov a,3;
```

- compilatorul va genera

```
mov dword ptr [...],3
```

```
int a;
_asm mov cx,a; //eroare, cx are 2 octeți
```

Indicatori de condiții

- instrucțiunea de atribuire nu modifică valorile indicatorilor de condiții
- aceștia își păstrează valorile pe care le aveau de la instrucțiunile anterioare

Utilizare practică (1)

```
int a,b;
```

- cum se traduce linia de cod

```
a=b;
```

- varianta greșită:

```
_asm mov a,b;
```

  – ambii operanzi sunt locații de memorie - eroare

Utilizare practică (2)

- varianta corectă:

```
_asm {
  mov eax,b
  mov a,eax
  }
```

- în general, pentru o instrucțiune C/C++ sunt necesare mai multe în limbaj de asamblare